Resource-Aware Collaborative Cloud IDE using Intelligent Optimization and Container Orchestration

BE Computer 2026–2027

Army Institute of Technology
Savitribai Phule Pune University
Computer Engineering

October 28, 2026

Under the guidance of **Prof. Yogita Hambir**

Contents

1. Introduction
2. Problem Statement
3. Gaps in Present Study
4. Proposed Methodologies
5. Resource Utilization
6. System Architecture
7. Conclusion

Introduction

- Modern software development is rapidly shifting toward cloud-based environments, enabling remote access and collaboration.
- Existing collaborative IDE platforms face high latency, synchronization conflicts, and inefficient resource utilization, especially under dynamic multi-user workloads.
- This project proposes a collaborative cloud IDE that integrates real-time programming with intelligent resource management.
- By leveraging containerization and orchestration, the system aims to achieve scalability, efficiency, and optimized performance.

Problem Statement

- Latency issues during real-time code synchronization
- Conflict handling complexity in multi-user editing
- Inefficient resource allocation, leading to wastage
- Lack of adaptive scaling mechanisms
- Poor handling of dynamic workloads and user spikes

Research Gaps

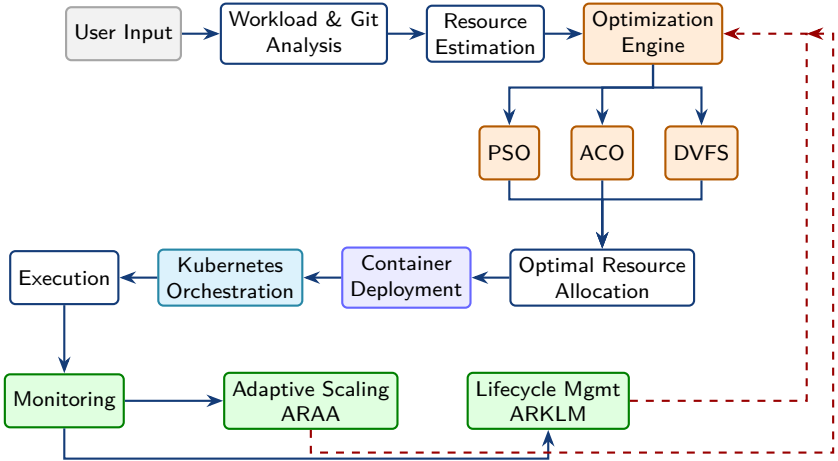
The present study identifies the following gaps:

- Lack of integration between real-time collaboration and resource optimization
- Limited use of intelligent scheduling algorithms in cloud IDEs
- Inefficient handling of multi-user synchronization at scale
- Absence of energy-aware and resource-aware execution models
- Dependency on static resource allocation strategies
- Limited automation in container lifecycle and workload management

Research Gaps: Literature Survey

Paper	Year	Methodology	Gaps
Adaptive Resource Allocation for Workflow Containerization on Kubernetes (Shan et al.)	2023	MAPE-K based adaptive resource allocation (ARAS), Kubernetes workflow engine, dynamic vertical scaling	No energy optimization, no intelligent optimization algorithms (ACO/PSO), lacks prediction capability, high monitoring overhead
A Hybrid Multi-Faceted Task Scheduling Algorithm for Cloud Computing Environment (Dubey & Sharma)	2023	Hybrid metaheuristic combining PSO and ACO, simulation using CloudSim	Not implemented in Kubernetes/container environment, no real-time scaling, no energy-aware optimization, only simulation-based validation
Adaptive Energy Optimization in Cloud Computing Through Containerization (Beena et al.)	2025	Integration of ACO, PSO and DVFS with Kubernetes, adaptive resource adjustment (ARAA), automated lifecycle management (ARKLM)	No ML-based prediction, limited proactive decision-making, lacks workload forecasting, scope for improved intelligent adaptability

Proposed Solution: Workflow



Dashed red arrows: closed-loop feedback to the Optimization Engine.

Resource Utilization

- Efficient use of computational resources such as CPU, memory, storage, and network in cloud environments.
- Demand varies dynamically due to:
  - Multiple concurrent users
  - Code execution workloads
  - Real-time collaboration activities
- Goal: maximize utilization while minimizing wastage and energy consumption.
- The system adopts an adaptive, optimization-driven resource utilization model.

Types of Resources Managed

CPU Resources Required for code execution, compilation, and processing

Memory (RAM) Used for runtime environments and active sessions

Storage Code files, logs, and container images

Network Bandwidth Real-time collaboration and data transfer

Container Resources Isolated execution environments for each user

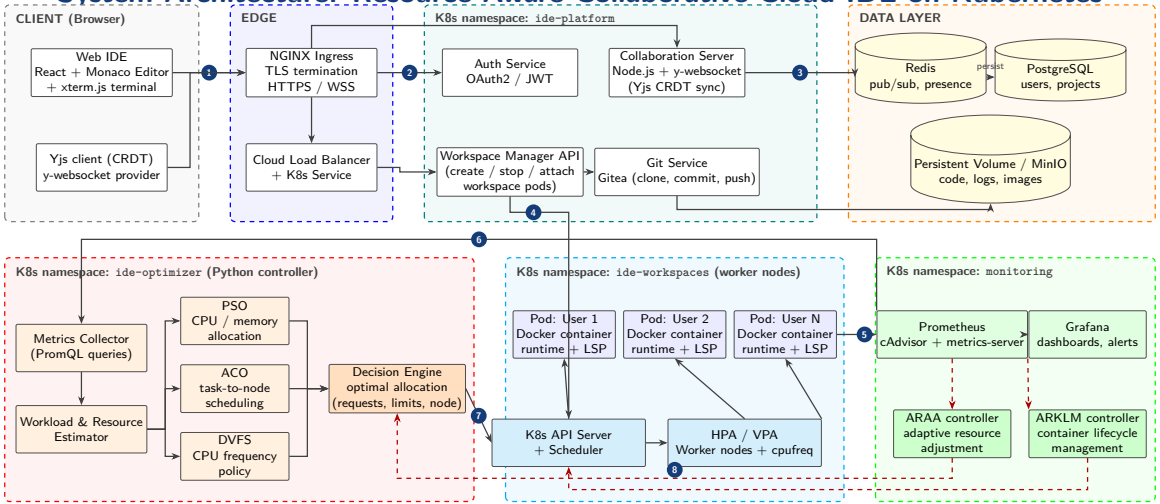
Optimization-Based Utilization

- **PSO** (Particle Swarm Optimization): finds optimal allocation of CPU and memory
- **ACO** (Ant Colony Optimization): efficient task scheduling across nodes
- **DVFS**: adjusts CPU power based on workload

Combined approach ensures

- Balanced load distribution
- Reduced energy consumption

System Architecture: Resource-Aware Collaborative Cloud IDE on Kubernetes



Flows: 1 HTTPS/WSS from browser 2 route to Auth, Collab, Workspace API 3 CRDT updates via Redis pub/sub, state in PostgreSQL 4 Workspace Manager asks K8s API to create a user pod 5 pod metrics scraped by Prometheus 6 Metrics Collector queries Prometheus 7 Decision Engine patches requests/limits and node placement; DVFS sets CPU frequency 8 ARAA/ARKLM feedback (MAPE-K loop, dashed).

Conclusion

- Real-time collaborative coding in the cloud using **Docker** for isolated environments and **Kubernetes** for scalability and load balancing.
- Intelligent optimization (PSO, ACO, DVFS) improves resource utilization and performance, enabling multiple users to work efficiently with low latency.
- Ensures reliability, scalability, and enhanced developer productivity, making it a modern and future-ready cloud IDE solution.

Thank You